

Oblivious Algorithms For Submodular Function Maximization

Destin Niyomufasha

August 2026

Abstract

Submodular maximization has been studied extensively, as it provides a generalized combinatorial framework for modeling a wide range of problems across multiple disciplines, including machine learning, economics, and social sciences, among others. Although significant progress has been made toward obtaining optimal approximation guarantees, many of the resulting algorithms are often too slow and impractical in settings involving massive datasets. This motivates the study of faster algorithms that achieve not necessarily optimal, but strong approximation guarantees with significant gains in runtime and query complexity.

In this work, we take a step towards understanding the query complexity lower-bounds, $\Omega(q(n))$, for algorithms achieving non-trivial $(\frac{1}{4} + \epsilon)$ approximation results. We generalize oblivious algorithms, which are local, $O(n)$ -query randomized algorithms that achieve ≥ 0.4853 approximation on the maximum directed cut problem, to submodular maximization and study their performance. On a finite ground set $\{1, \dots, n\}$, we define a submodular function g_n such that any oblivious algorithm is forced to select elements uniformly at random and show that $\lim_{n \rightarrow \infty} \alpha = \frac{\mathbb{E}[g_n]}{g_n(OPT)} \rightarrow 1/4$.

1 MaxDiCut and Oblivious Algorithms

Given a directed graph $G = (V, E, w : E \rightarrow \mathbb{R}_{\geq 0})$ with $n = |V(G)|$, a directed cut induced by $S \subseteq V$ is defined as an assignment $\mathbf{x} = (x_v)_{v \in V} \in \{0, 1\}^n$ such that $x_v = 1$ if and only if $v \in S$. The value of $\mathbf{x}$ in G , denoted $\partial_G^+(\mathbf{x})$ is

$$\sum_{i,j \in [n]} w_{ij} \mathbb{1}[x_i = 1 \wedge x_j = 0].$$

The maximum directed cut (MaxDiCut) problem asks, given a directed graph G , to find $\arg \max_{\mathbf{x} \in \{0,1\}^n} \partial_G^+(\mathbf{x})$. Similarly, the Maximum Cut (MaxCut), defined on undirected graphs, asks to find an assignment $\mathbf{x}$ maximizing

$$\sum_{i,j \in [n]} w_{ij} \mathbb{1}[x_i + x_j = 1]$$

MaxCut is a restriction of MaxDiCut in the sense that given an undirected graph, we can split an edge into two anti-parallel edges. Both of these problems are classic NP-complete problem with a huge literature about their (in)approximability, more notably [Goemans et al.](#), [Håstad](#), [Khot et al.](#), and [Lewin et al.](#)

The naive algorithm of randomly selecting $S \sim \mathcal{U}(V(G))$ achieves $\frac{1}{4}$ -approximation in expectation for MaxDiCut, and $\frac{1}{2}$ for MaxCut. Indeed, a weight w_{ij} edge is cut with $\frac{1}{4}$ and $\frac{1}{2}$ probabilities respectively. On the other hand, the best approximation ratio that can be obtained in polynomial time for the MaxCut problem is $\alpha_{CUT} = 0.8785$, obtained by the celebrated SDP-based

approximation algorithm of Goemans and Williamson, which later Khot et al. showed to be tight under Khot's Unique Games Conjecture (UGC) Khot et al. On a high level, the algorithm finds n -dimensional unit vectors $y_i \in S_n$ for each vertex to maximize $\frac{1}{2} \sum_{i \leq j} w_{ij}(1 - y_i y_j)$, $i, j \in [n]$ within ϵ additive error in polynomial running time dependent polynomially on $\log(1/\epsilon)$, and defines $S = \{i | i \in [n], y_i \cdot r \geq 0\}$ where r is the normal vector of a random hyperplane through the origin. The algorithm naturally leads to a $\alpha_{DICUT} = .79607$ -approximation, which Brakensiek et al. later improved to $\alpha_{Di-CUT} = 0.874$, and showed to be tight (up to a small δ) assuming Knot's UGC.

Motivated by the connection of the Cut problem with "local" and "distributed" models of computation, Feige et al. introduced a family of randomized algorithms for MaxDiCut called *oblivious* algorithms, which are *local* algorithms in the sense that they look at only the in- and out-degree of each vertex, and achieve non-trivial approximation $\alpha_{DICUT} \geq \frac{1}{4}$, by. On a high level, an oblivious algorithm defines k -step probability distribution (selection) function $p : [0, 1] \rightarrow [0, 1]$ and constructs a cut $\mathbf{x} \in \{0, 1\}^n$ by independently setting $\mathbf{x}_i = 1$ with probability $p(bias(v_i))$, where the *bias* of a vertex is:

$$bias(v_i) = \frac{\partial^+(v_i)}{\partial^+(v_i) + \partial^-(v_i)}. \quad (1)$$

When it holds for the selection function p that $p(a) = 1 - p(a)$, $a \in \mathbb{R}$, we call p antisymmetric. For antisymmetric functions, it is easy to see that oblivious algorithms' approximation guarantee is ≤ 0.5 . Indeed for a directed cycle $bias(v_i) = \frac{1}{2}$ and we can only pick randomly. Furthermore, oblivious algorithms have to be random since otherwise, in case $bias(v_i) = bias(v_j)$, $\forall i, j \in [n]$, either everything is picked, or nothing is picked.

Despite being extremely local, oblivious algorithms achieve non-trivial results. For example, the oblivious algorithm corresponding to the selection function

$$p(bias(v_i)) = \begin{cases} 0 & \text{if } 0 < bias(v_i) < 1/3, \\ 1/2 & \text{if } \frac{1}{3} \leq bias(v_i) \leq \frac{2}{3}, \\ 1 & \text{if } \frac{2}{3} < bias(v_i) < 1 \end{cases}$$

achieves a $\frac{3}{8}$ -approximation ratio.

Furthermore, Feige and Jozeph used showed a selection function and designed a factor-revealing linear program to find graphs on which the oblivious algorithm corresponding to the defined selection function performs the worst, showing approximation guarantee of ≥ 0.483 . At the same time, they showed that no oblivious algorithm can achieve $\alpha_{DICUT} > 0.4899$. Hwang et al. tightened this gap, obtaining an algorithm that achieves $\alpha_{DICUT} = 0.4853$. We summarize performance guarantees of oblivious algorithms on the MaxDicut problem below.

Theorem 1. *There is an oblivious algorithm with $\alpha_{DICUT} = 3/8$ approximation ratio.*

Proof. For some $0 < \delta < \frac{1}{2}$, consider the selection function:

$$p_\delta(bias(v)) = \begin{cases} 0 & \text{if } 0 < b_v < \delta, \\ 1/2 & \text{if } \delta \leq b_v \leq 1 - \delta, \\ 1 & \text{if } 1 - \delta < b_v < 1 \end{cases}$$

We denote $U^- = \{v \in V(G) | bias(v) < \delta\}$, $B = \{v \in V(G) | \delta \leq bias(v) \leq 1 - \delta\}$, and $U^+ = \{v \in V(G) | bias(v) > 1 - \delta\}$. We assume that there is no (u, v) such that $u \in U^-$ and $v \in U^+$ or vice versa. This is w.l.o.g since if such an edge, $e_i = \{u, v\}$, exists, we consider $G' = G(V + \{w\}, E' = E(G) - e_i + \{e_{i1}, e_{i2}\})$ where $e_{i1} = (u, w)$ and $e_{i2} = (w, v)$, such that $w(e_i) = w(e_{i1}) = w(e_{i2})$. Note that, by definition, w lies in B , and maximum cut is preserved. In

particular, if $e_i \in \partial_G^+$, then e_{i1} is cut. Otherwise, neither e_{i1} nor e_{i2} are cut.

We further transform the graph and consider $G' = G + G^T$. Observe that this operation preserves the approximation ratio, i.e. $\alpha_{DICT}(G) = \alpha_{DICT}(G')$ since both the expected and optimal cuts double. All subsequent claims are about G' . It follows that in G' , $U^- = U^+$. From now on we work with G' and abuse notation and denote it as G , and normalize weights so that $\sum_{ij \in E(G)} w_{ij} = 1$.

$$\text{Define } p = \sum_{ij \in E(G)} w_{ij} \cdot (\mathbb{1}_{[i \in U^-]} + \mathbb{1}_{[j \in U^-]}), ap = \sum_{ij \in E(G)} w_{ij} \cdot (\mathbb{1}_{[i \in U^-]} \cdot \mathbb{1}_{[j \in U^-]}), bp = \sum_{ij \in E(G)} w_{ij} \cdot (\mathbb{1}_{[i \in U^-]} \cdot \mathbb{1}_{[j \notin U^-]}), \text{ and } cp = \sum_{ij \in E(G)} w_{ij} \cdot (\mathbb{1}_{[i \notin U^-]} \cdot \mathbb{1}_{[j \in U^-]}).$$

We have that $2a + b + c = 1$, $a + b \leq \delta$, and the weight of endpoints in B is $2 - 2p$. Furthermore, $\mathbb{E}[\partial^+] = bp + \frac{1-2p(a+b+c)}{4} = p(1-a) + p(a+c) + \frac{1-2p(1-a)}{4}$. In words, edges from U^- to B and from B to U^+ , both whose total weight is bp each, are cut with $\frac{1}{2}$. Edges inside B are cut with $\frac{1}{4}$ probability, and by definition, their net weight is exactly $1 - 2(a + b + c)$.

We say that the endpoint u (resp. v) to the edge (u, v) is misplaced by a cut $(S, V \setminus S)$ if $u \in S$ (resp. $v \notin S$). It follows from the definition of p_δ and B , that regardless of whether the node $v \in B$ is picked or not δ fraction of its endpoints are misplaced. Hence $\delta(2 - 2p)$ total endpoint weight, is misplaced. Furthermore, since U^+ is picked and U^- is not, $2(a + c)p$ endpoint weight is misplaced. It follows then that $\partial_{OPT}^+ \leq 1 - \frac{(2-2p)\delta + 2(a+b)p}{2} = 1 - (1-p)\delta + (a+b)p$. Analyzing α_{DICT} which is at least the ratio between (1) and (2), it's not hard to see that having $a + c = \delta$ maximized α_{DICT} . Maximizing the resulting α_{DICT} function, one can observe that α_{DICT} is max when $\alpha = \frac{1}{3}$, in which case $\alpha_{DICT} = \frac{3}{8}$.

□

Theorem 2. *There is a submodular function achieving $\alpha_{DICT} = 0.483$.*

Proof. For a given n -step selection function f , we use linear programming to find the graph on which the corresponding oblivious algorithm performs the least. We denote the n intervals as (b_{i-1}, b_i) .

Define $V_i = \{v \in V(G) | b_{i-1} \leq \text{bias}(v) \leq b_i\}$, and $S_i = OPT \cap V_i$ and $S_{i+n} = V_i \setminus S_i$. For every $i, j \in [2n]$, denote $e_{ij} = \{(u, v) \in E(G) \mid u \in S_i \text{ and } v \in S_j\}$. The linear program which minimizes $\sum_{i,j \in [2n]} e_{ij} \cdot f(b_i) \cdot (1 - f(b_j))$, where for $i \geq n + 1$, $f(b_i) = f(b_{i-n})$, with respect to:

- $e_{ij} \geq 0, \forall i, j \in [2n]$
- $\sum_{i \in [n], j \in [n+1, \dots, 2n]} e_{ij} = 1$
- $b_{i-1} \leq \frac{\sum_{j \in [2n]} e_{ij}}{\sum_{j \in [2n]} e_{ij} + e_{ji}} \leq b_i, \forall i \in [n]$

Consider the selection function f defined on unit interval:

$$f(x) = \begin{cases} 0 & x < \frac{1}{4} \\ \frac{1}{4} + 0.01 \cdot i & \frac{1}{4} < \frac{1}{4} + 0.005i \leq x \leq \frac{1}{4} + 0.05(i+1), 0 \leq i < 50 \\ \frac{1}{2} & x = \frac{1}{2} \\ \frac{1}{4} + 0.01 \cdot j & \frac{1}{4} < \frac{1}{4} + 0.005j \leq x \leq \frac{1}{4} + 0.05(j+1), 50 \leq j < 100 \\ 1 & b_i > \frac{3}{4} \end{cases}$$

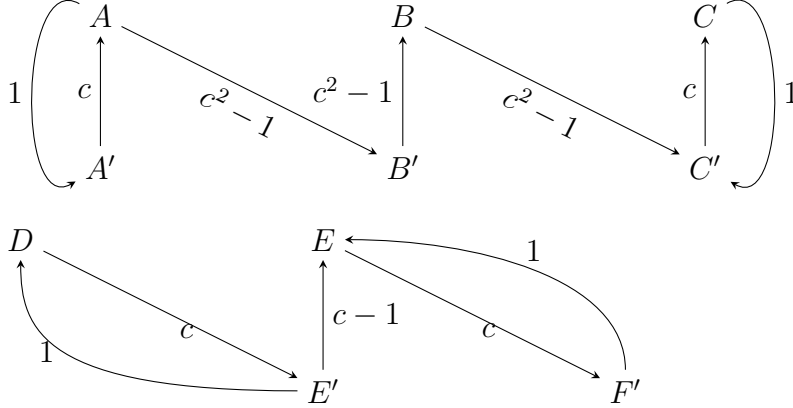
Running the LP on f achieves 0.483

□

Theorem 3. *There is a graph on which every oblivious algorithm achieves at most 0.4899.*

Proof. We construct graphs G_1 and G_2 and consider

$$G = G_1 \cup G_2 \cup G_2 \cup G_2.$$



Define $\gamma = f(\frac{c}{c+1})$. For G_1 , we have that $\text{bias}(A) = \text{bias}(A') = \frac{c}{c+1}$, $\text{bias}(B) = \text{bias}(B') = \frac{1}{2}$, and $\text{bias}(C) = \text{bias}(C') = \frac{1}{c+1}$, and $\partial_{OPT}^+ = 2C^2$. It follows that $\alpha_{DICT}(G_1) = \frac{2\gamma(1-\gamma)(1+c) + (\frac{1}{4}+\gamma)(c^2-1)}{2c^2}$. For G_2 , we have $\text{bias}(D) = \frac{c}{c+1}$, $\text{bias}(F) = \frac{1}{c+1}$, $\text{bias}(E) = \text{bias}(E') = \frac{1}{2}$, and $\partial_{OPT}^+ = 2c$. It follows that $\alpha_{DICT}(G_2) = \frac{1+(\gamma+\frac{1}{4})(c-1)}{2c}$. It follows then that $\alpha_{DICT}(G) = \frac{(\frac{1}{4}+\gamma)(c^2+6c-6)}{2c^2+6c}$ [complete the function]. For a fixed c , $\alpha_{DICT}(G)$ is parabolic in γ , and for $c = 1.25$, in particular, it achieves max at $\gamma = 0.4899$ $\square$

2 Submodular function maximization

Given a ground set $N = \{1, \dots, n\}$, and a function $f : 2^N \rightarrow \mathbb{R}_{\geq 0}$, define $f(e|S)$ for some $e \in N$ and $S \subseteq N$ to be e 's marginal to S with respect to f . i.e. $f(e|S) = f(S \cup \{e\}) - f(S)$. f is submodular if, for $A \subseteq B \subseteq N$, and $e \in N \setminus B$, $f(e|A) \geq f(e|B)$, or equivalently $f(A) + f(B) \geq f(A \cup B) + f(A \cap B)$. Put simply, a set function f is submodular if it has diminishing marginals. It is monotone if it has non-negative marginals, i.e. $f(e|S) \geq 0, \forall e \in N, S \subseteq N$. In this work we consider the maximization problem for general submodular function. However, we note that the problem is often studied under many different constraints, including the knapsack constraint in which one is a given weight function $w : N \rightarrow \mathbb{R}$ and a budget $W \in \mathbb{R}$ and is asked to find the subset S with $w(S) = \sum_{e \in S} w(e) \leq W$ such that $f(S)$ is maximized, and the matroid constraint where the subset S return has to be an independent set in the matroid $(N, \mathcal{I})$, among others. It would be an interesting line of work to the performance of oblivious algorithms under these constraints.

Lemma 2.1. *For any sequence of sets $S_i \subseteq N, i \in [n]$, and a submodular function $f : 2^N \rightarrow \mathbb{R}_{\geq 0}$, $f(\bigcup S_i(p_i)) \geq \sum_{b \in \{0,1\}^n} \prod p_i^{b_i} (1 - b_i)^{(1-p_i)} f(\bigcup_{i:b_i=1} S_i)$.*

Proof. To prove the lemma for $\{S_1\}$, we proceed by induction on its elements. For $\emptyset$ the claim follows trivially. Write $S_1 = S'_1 \cup \{x\}$ for some $x \in S_1$. By submodularity, $f(S_1(p_1)) \geq f(S'_1(p_1)) + f(S_1(p) \cup S'_1) - f(S'_1)$

$$\begin{aligned} \mathbb{E}[f(S_1(p_1))] &\geq \mathbb{E}[f(S'_1(p_1))] + \mathbb{E}[f(S_1(p) \cup S'_1) - f(S'_1)] \\ &\geq (1 - p_1)f(\emptyset) + p_1f(S'_1) + p_1(f(S_1) - f(S'_1)) \\ &= (1 - p_1)f(\emptyset) + p_1f(S_1) \end{aligned}$$

Fix $X = \bigcup_{i \in [n-1]} S_i(p_i)$, and define $g(S) = f(X \cup S)$. It's not hard to see that g is submodular, and by the claim above, $\mathbb{E}[g(S_n(p_n))] \geq (1-p_n)f(\emptyset) + p_nf(S_n)$. By not conditioning on $\bigcup_{i \in [n-1]} S_i(p_i) = X$,

$$\begin{aligned} \mathbb{E}[f(\bigcup_{i \in [n]} S_i(p_i))] &= (1-p_n)\mathbb{E}[f(\bigcup_{i \in [n-1]} S_i(p_i))] + \mathbb{E}[p_nf((\bigcup_{i \in [n-1]} S_i(p_i)) \cup S_n)] \\ &\geq (1-p_n) \sum_{i \in [n-1]} \prod_{b \in \{0,1\}^{(n-1)}} p_i^{b_i} (1-p_i)^{(1-b_i)} f(\bigcup_{i: b_i=1} S_i) + p_n(f((\bigcup_{i \in [n-1]} S_i(p_i)) \cup S_n)) \\ &= \sum_{i \in [n]} \prod_{b \in \{0,1\}^n} p_i^{b_i} (1-p_i)^{1-b_i} f(\bigcup_{i: b_i=1} S_i) \end{aligned}$$

□

Theorem 4. Let f be a submodular function, $OPT = \max_{S \subseteq N} f(S)$, and R a uniformly random set. Then $\mathbb{E}[f(S)] = \frac{1}{4} \cdot OPT$. Furthermore, if f is symmetric, i.e. $f(S) = f(N \setminus S)$, $\mathbb{E}[f(S)] = \frac{1}{2}OPT$.

Proof. By 2.1 $f(N(\frac{1}{2})) = (OPT)(1/2) \cup (N \setminus OPT)(\frac{1}{2}) \geq \frac{1}{4}f(\emptyset) + \frac{1}{4}f(OPT) + \frac{1}{4}f((N \setminus OPT)) + \frac{1}{4}f(N)$ □

Corollary 1. For a directed graph G , $\mathbb{E}[\partial_S^+] = \frac{1}{4}OPT$.

3 Limitations of oblivious algorithms on submodular maximization

It is well-known that MaxDicut is a submodular maximization problem. Indeed given cuts $(S_1, V \setminus S_1)$ and $(S_2, V \setminus S_2)$ such that $S_1 \subseteq S_2$ the marginal cut gain for adding a new vertex v in S_2 is smaller than in S_1 , since there are at most as much endpoints from v in S_1 as in S_2 .

Furthermore, oblivious algorithms are well defined on set functions. Indeed, it suffices to write a selection p function that uniformly samples elements with $p(bias(e))$, where

$$bias(e) = \frac{f(\{e\})}{f(\{e\}) + f(N \setminus \{e\})},$$

So, how powerful are they?

Theorem 5. Given $N = \{1, \dots, n\}$ for some $n \in \mathbb{Z}_+$, there exists a bi-variate function $f_n : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ such that the function $g_n(S) = f_n(|S \cap OPT|, |S \cap (N \setminus OPT)|)$ is submodular. and

$$\lim_{n \rightarrow \infty} \frac{\mathbb{E}[g_n]}{g_n(OPT)} = \frac{1}{4}$$

where Define $\mathbb{E}[g_n] = \frac{1}{2^n} \sum_{x=0}^k \sum_{y=0}^m \binom{k}{x} \binom{m}{y} f_n(x, y)$. Furthermore, the function g_n is singleton symmetric. i.e. $g(\{e\}) = g(N \setminus \{e\})$, $\forall e \in N$, and $g_n(OPT) = f_n(\lfloor \frac{n}{2} \rfloor, 0)$

Corollary 2. $\forall \epsilon > 0$, no oblivious algorithm can achieve $(\frac{1}{4} + \epsilon)$ -approximation on the submodular maximization problem.

In other words, one can define a submodular function for which the value of $S \subset N$ is defined by the order of $S \cap OPT$. On a high level, this function is linear under and over OPT and $\overline{OPT}$, and hyperbolic otherwise.

Proof. For $n \in \mathbb{Z}_+$, let $k = \lfloor \frac{n}{2} \rfloor$ and $m = \lceil \frac{n}{2} \rceil$. Define

$$\begin{aligned} f_n(x, 0) &= \frac{m}{k}x & 0 \leq x \leq k \\ f_n(x, m) &= \frac{x}{m-1} & 0 \leq x \leq k \\ f_n(0, y) &= \frac{m-y}{m-1} & 0 \leq y \leq m \\ f_n(k, y) &= m-y & 0 \leq y \leq m \\ f_n(x, y) &= \frac{m-y}{m-1} + x(b_1 + dy) & 1 \leq x \leq k-1, 1 \leq y \leq m-1 \end{aligned}$$

where

$$b_1 = f_n(1, 1) - f(0, 1) = \frac{m-2}{k}, d = \frac{\frac{1}{m-1} - b_1}{m-1}$$

We show that $\lim_{n \rightarrow \infty} \frac{\mathbb{E}[f_n]}{f_n(k, 0)} = \frac{1}{4}$.

Let $X \sim \text{Bin}(k, \frac{1}{2})$ and $Y \sim \text{Bin}(m, \frac{1}{2})$ be independent. Then the weighing in the double sum implies

$$\mathbb{E}[f_n] = \mathbb{E}[f_n(X, Y)]$$

The boundary events $\{X \in \{0, k\}\} \cup \{Y \in \{0, m\}\}$ have probability $O(2^{-k} + 2^{-m})$, so their total contribution to $\mathbb{E}[f_n]$ goes to 0 as $n \rightarrow \infty$; hence

$$\mathbb{E}[f_n] = \mathbb{E}\left[\frac{m-Y}{m-1} + X(b_1 + dY)\right].$$

Using $\mathbb{E}[f_n] = \frac{k}{2}$, $\mathbb{E}[f_n] = \frac{m}{2}$, and $\mathbb{E}[XY] = \frac{km}{4}$,

$$\mathbb{E}[f_n] = \frac{m - \frac{m}{2}}{m-1} + \frac{km}{4}d = \frac{m}{2(m-1)} + \frac{k}{2}b_1 + \frac{km}{4}d$$

Substituting $b_1 = \frac{m-2}{k}$ and $d = \frac{\frac{1}{m-1} - b_1}{m-1}$, we get

$$\mathbb{E}[g_n] = \frac{km + m^3 - 3m^2 + 6m - 4}{4(m-1)^2} \approx \frac{m}{4} \text{ (by } n \rightarrow \infty \text{)}$$

Since $g_n = m$,

$$\frac{\mathbb{E}[f_n]}{f_n(k, 0)} \rightarrow \infty$$

as $n \rightarrow \infty$

□

References

- Brakensiek, Joshua et al. (2022). *Separating MAX 2-AND, MAX DI-CUT and MAX CUT*. arXiv: [2212.11191 \[cs.CC\]](#).
- Feige, Uriel and Shlomo Jozeph (2015). “Oblivious Algorithms for the Maximum Directed Cut Problem”. In: *Algorithmica* 71.2. Preliminary version: arXiv:1010.0406, pp. 409–428. doi: [10.1007/s00453-013-9806-z](#).

- Goemans, Michel X. and David P. Williamson (1995). “Improved Approximation Algorithms for Maximum Cut and Satisfiability Problems Using Semidefinite Programming”. In: *Journal of the ACM (JACM)* 42.6, pp. 1115–1145. DOI: [10.1145/227683.227684](https://doi.org/10.1145/227683.227684).
- Håstad, Johan (2001). “Some Optimal Inapproximability Results”. In: *Journal of the ACM* 48.4, pp. 798–859. DOI: [10.1145/502090.502098](https://doi.org/10.1145/502090.502098).
- Hwang, S., Noah G. Singer, and S. Velusamy (2024). *Oblivious Algorithms for Maximum Directed Cut: New Upper and Lower Bounds*. arXiv: [2411.12976](https://arxiv.org/abs/2411.12976) [cs.DS]. URL: <https://arxiv.org/abs/2411.12976>.
- Khot, Subhash et al. (2007). “Optimal Inapproximability Results for MAX-CUT and Other 2-Variable CSPs”. In: *SIAM Journal on Computing* 37.1, pp. 319–357.
- Lewin, Michael, Dror Livnat, and Uri Zwick (2002). “Improved Rounding Techniques for the MAX 2-SAT and MAX DI-CUT Problems”. In: *Proceedings of the 9th International Conference on Integer Programming and Combinatorial Optimization (IPCO)*. Vol. 2337. Lecture Notes in Computer Science. Springer, pp. 67–82. DOI: [10.1007/3-540-47867-1_6](https://doi.org/10.1007/3-540-47867-1_6).